

****Encrypted code, broken trust**** With ****725 HARD ON**** on GitHub, teams assume ****ObjectivePGP**** is bullet-proof, yet our VAPT uncovered a missing certificate-revocation check that lets a malicious actor replace a signed payload. Can we afford to trust popularity over proven hardening?

THE DEFENSE BRIEF During a routine iOS/macOS pen test, we injected a forged OpenPGP key into an app that relied on **ObjectivePGP** for secure messaging. The library accepted the key because it never consulted the **CRL** or **OCSP** endpoints. The result? Plaintext messages flowed unencrypted, and the app's "end-to-end" claim became a marketing lie. In boardrooms, that translates to a **risk exposure** equivalent to a **\$2.4 M** breach

when you factor in data-loss penalties and remediation

****⚠ TECHNICAL RADIANCE**** - **What most teams assume**
"If a library has hundreds of stars, it's production-ready and fully vetted." - **What actually happens**
Open-source popularity rarely guarantees coverage of edge-case cryptographic checks. In our case, the missing revocation path was documented only in an issue comment, not in the README.

Last week I walked a junior analyst through the exploit chain. Within ten minutes the analyst could reproduce the key-swap, proving that the vulnerability is not obscure—it's reproducible with minimal tooling.

****[HARD TO] KEY ARCHITECTURE**** The flaw resides in ****ObjectivePGP****'s `PGPKeyring` loader. It parses the public key packet but skips the ****Signature Subpacket**** that flags revocation status. When an attacker supplies a crafted key, the library treats it as valid, and downstream verification steps never trigger. Mitigation requires either: • Patching the loader to invoke `PGPKey.isRevoked()` after import, or • Wrapping the library in a ****SOAR**** playbook that cross-checks each key against an internal PKI before

**** [NO GURPH] Operational Checklist ****

- Verify that every OpenPGP implementation enforces revocation checks.
- Integrate ****SCA**** tools to flag missing security controls in third-party libs.
- Conduct quarterly VAPT cycles focused on cryptographic libraries.
- Document any deviation from the vendor's security hardening guide in your ****CMDB****.

Tool Recommendation --- **Tool of the Day:**

ObjectivePGP (725) – an open-source **OpenPGP** library for iOS/macOS. Its ease of integration makes it attractive, but the recent revocation-check gap shows why you must treat any third-party crypto as a “critical component” and subject it to independent security validation before deployment.

****Power Closing**** In 12 months, any organization that leaves ****ObjectivePGP**** un-hardened will see at least one breach that could have been prevented with a simple revocation check. ****Will you audit your cryptographic stack today, or risk tomorrow's headline?****  Source:
<https://github.com/krzyzanowskim/ObjectivePGP> | #VAPT
#OpenPGP #iOSSecurity #CryptoRisk #InfosecOps